

Git Spiel

1. Zuerst habe ich das git quest repo neu geklont, wie in der Aufgabe verlangt. Dann habe ich mit `git diff master end` herausgefunden, welche Dateien in end geändert wurden, damit ich weiß, welche Dateien ich nicht ändern soll im master branch. Ich habe dann die Datei `hero.md` geändert, einfach eine kleine zeile hinzugefügt, gespeichert, geadded und dann committet. Dann habe ich den Befehl `git merge end` ausgeführt und die Aufgabe war erledigt. Es gab keinen merge Konflikt.
2. Aufgabe 2 war relativ ähnlich, da musste ich lediglich darauf achten, eine andere stelle zu ändern als im end branch, habe ich gemacht und es lief genauso wie bei aufgabe1, nur mit dem unterschied, dass sich ein Fenster für eine merge message geöffnet hat.
3. bei Aufgabe 3 musste ich dann eine gleiche Zeile änder wie in im end branch. Als ich dass dann gemacht habe, nachdem ich das git quest repo neu geklont habe und dann mergen wollte kam ein merge Konflikt (**CONFLICT (content): Merge conflict in questlog.md**) bei raus. Diesen kann man ganz leicht beheben, indem man in die Datei geht und dort die zeichen wegmacht und den Teil, den man da raus haben will auch entfernt. Wenn man bei dem master branch die gleiche Änderung macht, dann entsteht kein merge Konflikt und es funktioniert ganz normal.
4. Aufgabe 4 habe ich so verstanden, dass ich zuerst eine Datei ändern soll, die in end auch verändert wurde, aber halt an einer anderen stelle, dann adden und dann commiten. Dann den end branch auf das Level von master bringen , also master in end mergen und dann danach auf den master branch wechseln und das gleiche umgekehrt machen. Also dann end in master mergen. Als ich dann master in end gemerged habe öffnete sich nano, damit ich eine merge message angebe und dann musste ich nur speichern und schließen. Dann wie gesagt auf master wechseln und end in master mergen. Das war dann ein fast forward merge ohne Konflikte etc.

Katzencafe

Warum sind die von Ihnen fomulierten Testfälle relevant?

Bei den Testfällen werden die wichtigen Funktionen der CatCafe Klasse überprüft um zu sehen, dass diese korrekt ablaufen. Man hat Test wie, ob Katzen korrekt hinzugefügt werden, ob die Anzahl der Katzen stimmt und ob man Katzen anhand des Namen oder Gewichts finden kann. Außerdem werden auch Fehltests getestet um zu sehen, ob es auch dahingehend funktioniert wie die Suche nach nicht vorhandenen Katzen.

Warum sind die formulierten Testfälle unterschiedlich?

Bei den Test werden unterschiedliche Fälle abgedeckt. Bei der einen wird das hinzufügen einer oder mehrere Katzen überprüft, bei der anderen das Verhalten eines leeren Cat Cafes. Außerdem gibt es auch Tests für erfolglose und erfolgreiche Suchanfragen. Das heißt es wird auch geprüft, wie es sich verhält, wenn es fehlschlägt. Die Test unterscheiden sich also in Verhalten und Eingabedaten.

https://github.com/JohannMagBackfisch/Jo_prog2_ybel_catcafe.git